# Diseño de microservicios

Tarea 4 · Proyecto: API de productos con frontend en React

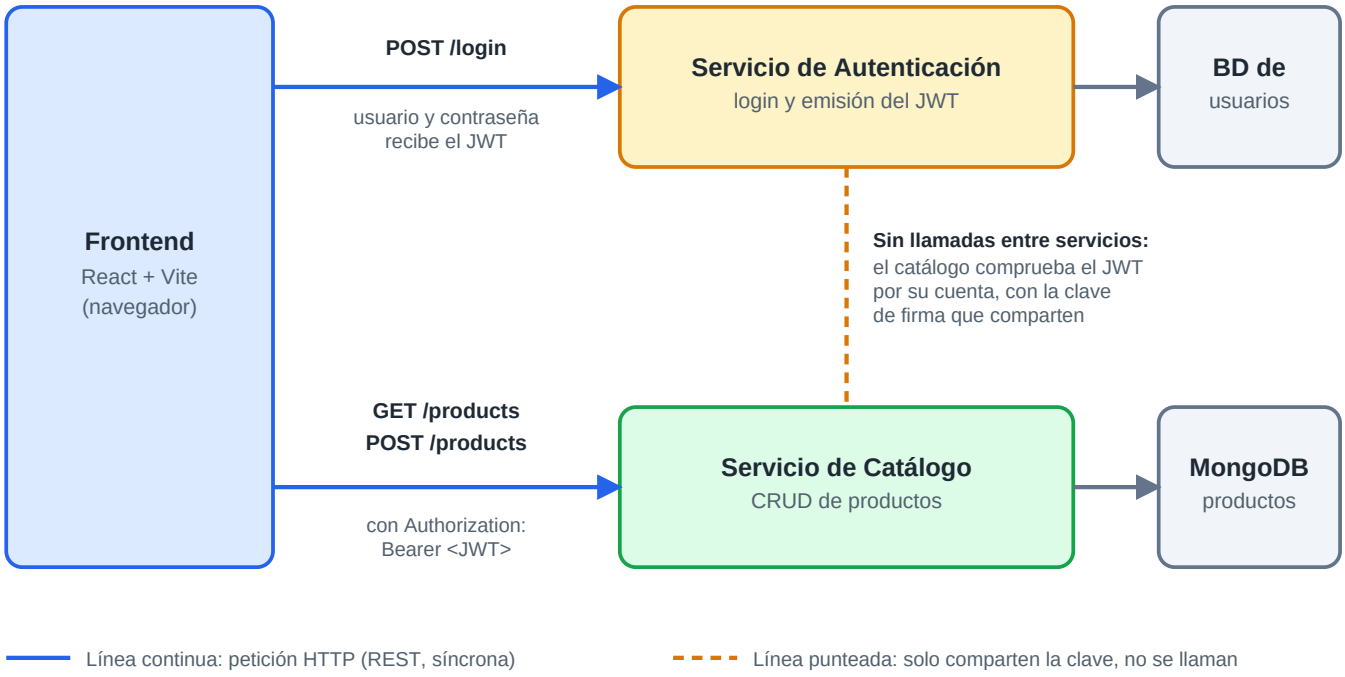
## 1. Servicios identificados

El proyecto ya está separado por dominios desde el refactor de la Tarea 3 (`backend/src/auth` y `backend/src/products`), así que los límites naturales salen de ahí. Propongo dos servicios:

| Servicio | Responsabilidad | Datos propios | Hoy vive en |
|---|---|---|---|
| **Autenticación** | Verificar usuario y contraseña, emitir el JWT (`POST /login`) y, más adelante, registro y gestión de usuarios. | Usuarios (hoy hay un usuario fijo en el código). | `backend/src/auth` |
| **Catálogo de productos** | CRUD de productos, validación con Zod y persistencia. Para crear exige un JWT válido. | Productos (MongoDB). | `backend/src/products` |

El frontend no es un servicio: es el cliente que consume a los dos.

## 2. Diagrama



## 3. Cómo se comunican y por qué

**Frontend con los servicios: REST síncrono (HTTP + JSON).** En login, listar y crear, la persona espera la respuesta en pantalla (¿entró?, ¿se guardó?). Es el caso típico de petición y respuesta inmediata, y REST es lo que el equipo ya usa y sabe probar con curl o Thunder Client.

**Entre servicios: ninguna llamada directa.** El JWT es autocontenido: el servicio de Autenticación lo firma y el de Catálogo comprueba la firma por su cuenta, sin preguntarle nada a Autenticación. Así el catálogo sigue atendiendo aunque Autenticación esté caído (la lectura es pública y la escritura solo falla si el token no es válido) y se evita una llamada de red extra en cada petición. Para que esto funcione, ambos servicios necesitan compartir la clave de firma (hoy `JWT_SECRET`); en un entorno real sería mejor firmar con una clave privada y verificar con la pública, así el catálogo no puede emitir tokens.

**Por qué no mensajería asíncrona (por ahora).** La mensajería (colas o eventos, por ejemplo RabbitMQ o Kafka) sirve cuando una tarea puede esperar o cuando varios servicios deben enterarse de un hecho. Hoy no hay ninguna: nada de lo que pasa en un servicio necesita avisar al otro en segundo plano, y agregar un intermediario sumaría complejidad sin beneficio. Tendría sentido si el proyecto suma, por ejemplo, un servicio de pedidos o notificaciones que deba reaccionar a eventos como «producto creado» o «stock bajo»; ahí conviene que el catálogo publique el evento sin quedar atado a quien lo recibe.

## 4. ¿Qué pasaría si NO se divide?

A medida que el proyecto crece, un único programa que lo hace todo se vuelve cada vez más difícil de cambiar y de sostener. Un cambio pequeño en el login obliga a volver a desplegar también el catálogo, y un error o un pico de tráfico en una parte puede tumbar la aplicación entera. No se puede escalar solo lo que más se usa (por ejemplo, las lecturas del catálogo) sin duplicar todo lo demás. Con el tiempo el código de usuarios y el de productos se enreda, varias personas se pisan al editar los mismos archivos y todo queda atado a una única base de datos y a las mismas tecnologías. Dividir en servicios con límites claros pone un freno a ese acoplamiento.

Dividir también tiene un costo (más piezas que desplegar, vigilar y comunicar), por eso hoy el proyecto, ya organizado por dominios, aguanta bien como una sola aplicación. La división se justifica cuando el crecimiento hace que los problemas anteriores duelan más que ese costo.